HybridRAG: Enhanced Knowledge Retrieval through Hybrid Integration of Knowledge Graphs and Vector Search Techniques

Ilia Bukin
Queen Mary University of London

***Abstract*—This paper explores the implementation of hybrid Retrieval-Augmented Generation (RAG) techniques to develop a robust conversational agent capable of delivering contextually relevant responses. The ensemble approach called Hybrid RAG proposed in this paper uses knowledge graphs (KGs) in conjunction with vector-based methods to enhance the accuracy of data retrieval. KG-based RAG methods, collectively dubbed Graph RAG, offer a structured way to represent interconnected data, facilitating a deeper understanding of relationships and hierarchies within the information. Similarly, vector-based methods (Naive RAG) use chunks of unstructured data, capturing nuanced contextual similarities through their embeddings. This paper presents a detailed analysis of the individual methods, highlighting how their combined strengths lead to superior retrieval and question-answering performance. We use two datasets, a generic Wikipedia page and domain-specific articles, to evaluate the Naive, Graph, and Hybrid RAG approaches on their retrieval and answer generation performance. The results indicate that while Hybrid RAG can generate accurate and contextually relevant answers, it did not outperform Naive RAG in our testing. Despite this, our method shows potential for further development. With further optimisation and more comprehensive testing, hybrid RAG could live up to its promise of delivering a more grounded and practical approach. Additionally, our paper introduces a conversational platform called GraphBot, which effectively showcases the capabilities of the RAG techniques discussed. This research offers valuable contributions to the growing body of knowledge in artificial intelligence by delivering valuable insights and solutions that can inspire and guide the development of next-generation conversational agents.**

***Index Terms*—Retrieval-Augmented Generation (RAG), Conversational agents, Vector Search, Knowledge Graphs, Indexing Methods**

I. Introduction

The advancements in the field of generative artificial intelligence (GenAI), specifically the advent of transformer-based models, have paved the way for the development of LLMs capable of generating human-like text and understanding context with high precision (Vaswani et al., 2023). Cutting edge LLMs such as Generative Pre-trained Transformer (GPT), which contain billions of parameters and have been trained on vast amounts of knowledge, are capable of answering questions and delivering accurate responses across a diverse range of domains, they fall short at addressing highly specialised questions or previously unseen knowledge they have not been explicitly trained on (Lewis et al., 2021).

This is where Retrieval-Augmented Generation (RAG) techniques have been widely used to combine LLMs' versatility with retrieval-based systems' accuracy. RAG aims to address some of the limitations of LLMs, including their inability to readily expand their memory, their tendency to produce "hallucinations" when confronted with unknown information, and the opaque nature of their responses. RAG opened possibilities to deploy GenAI models in a professional setting, such as in legal, medical, and financial sectors that require performing knowledge-intensive tasks on external knowledge sources where the accuracy of model responses is paramount. However, current implementations of RAG are not perfect. Common chunk-based retrieval approaches like NaiveRAG face issues such as missing or irrelevant content during extraction, incorrect information specificity and truncation due to hitting the LLM content window (Barnett et al., 2024). Another concern is RAG performance on tasks such as Query Focused Summarisation (QFS), where queries can be directed to the entire corpus. (Edge et al., 2024) It requires striking the right balance between covering all necessary information and maintaining depth for each topic. These issues become more apparent as the data volumes increase, posing significant challenges for effectively deploying RAG systems at scale.

A promising solution involves storing information in structured knowledge graphs (KGs). KGs provide a robust framework for storing and accessing vast amounts of data across multiple documents, storing document structures and their entities in a graph database like Neo4j. Lastly, employing multi-hop reasoning pipeline, such as reasoning and acting (ReAct), can further enhance the adaptability and specificity of the retrieval process (Yao et al., 2023). Multi-hop retrieval allows the RAG system to follow a chain of thought across multiple retrieval paths, synthesising information from various sources to form the final answer.

To address the current limitations of RAG systems, this paper develops a promising hybrid RAG system, combining vector and graph-based retrieval methods. The aim is to merge the nuanced semantic information captured by vector embeddings with the structured data of Knowledge Graphs. As a result, this paper presents a conversational platform that is capable of dynamic reasoning and meaningful engagement with users, inspiring new possibilities in the field of artificial intelligence.

II. Related Work

A. Vector Embeddings

Vector embedding is a method for representing individual features in data as numerical vectors in a continuous space. Various techniques, including Word2Vec, GloVe, and others, have been developed to create these representations from text. Self-attention mechanisms of transformer models like BERT and GPT offer a significant advancement over earlier methods (Vaswani et al., 2023). They can provide embeddings at multiple levels of granularity —ranging from words to entire documents. Additionally, these models are pre-trained on large and diverse corpora, simplifying their implementation without training and fine-tuning. In this paper, the embeddings are generated using the "text-embedding-3-small" model by OpenAI because it produces high-quality contextual embeddings where the representation of features like words changes based on their surroundings. Thus, vector embeddings facilitate similarity searches, which are explored below.

B. Vector Similarity Search Techniques

Vector similarity aims to measure how similar or different one or more vectors are to each other. This technique is fundamental to performing a similarity search between embeddings of the query and stored pieces of information in the vector database or a knowledge graph. There are several popular metrics to calculate said similarity.

- **Dot Product:** The simplest similarity metric is the dot product, which computes the sum of the products of the corresponding components of two vectors. The dot product can be affected by both the length and direction of the vectors; thus, without normalization, longer vectors can produce higher dot product values, even when the vectors are not well-aligned in direction (Pinecone, 2023). The dot product is calculated as follows:

$$A \cdot B = \sum_{i=1}^{n} A_i B_i \tag{1}$$

  where A and B are the two vectors, and A_i and B_i are the components of these vectors.
- **Cosine Similarity:** Cosine similarity measures the cosine of the angle θ between two vectors (i.e., their direction) and is unaffected by their size. This makes it especially useful for calculating the semantic similarity of text embeddings with varying lengths:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|} \tag{2}$$

  where $A \cdot B$ is the dot product of the vectors, and $\|A\|$ and $\|B\|$ are their magnitudes.
- **Other Metrics:** Euclidean distance can also measure similarity. However, it is also sensitive to magnitude and does not consider the directions of vectors. Thus, the cosine similarity method was chosen for this project.

C. Fundamentals of RAG System

The seminal work by Lewis et al. (2020) first introduced the method of using external knowledge for the expansion of capabilities of LLMs and coined the concept of RAG. Their architecture comprised two main components: a retrieval module and a generation module, with an augmentation step in between.

- The retriever, as the name suggests, oversees the search and extraction of information relevant to the user query. This component uses various retrieval techniques to search through a knowledge base. This paper focuses on vector and graph databases.
- The augmentation stage involves organising retrieved information and then combining it with the user query.
- The generator is a pre-trained LLM responsible for outputting the final answer based on the augmented input.

D. Naïve RAG

Naïve RAG has been extensively researched, and its performance documented. It first splits raw textual data into chunks to accommodate LLM's limited content window. These chunks are then encoded into vector representations using an embedding model and stored in a vector database. At query time, the system embeds the user question in the same vector space and then uses a retriever to perform the similarity search on chunks. The retriever then returns the top-ranked chunks, which are semantically similar to the query. This information is then augmented with the original prompt and sent for generation by an LLM. Barnett et al (2024) and others outline several limitations associated with RAG based on vector similarity search.

- Storing chunks of text for retrieval lacks granularity and cannot selectively target relevant information within them (Gao et al., 2024).
- Vector representations lack explainability as they store data in multi-dimensional space and do not provide transparent reasoning behind associations. This opacity makes it challenging to understand why certain chunks were retrieved over others.
- Parsing a large amount of unstructured information into the LLM context window is inefficient at best, and at worst, it can result in the truncation of critical information and potential loss of context due to the "lost in the middle" problem (Liu et al., 2023).
- A single retrieval process may need to be revised to gather adequate context for answering complex queries. If the initial retrieval misses key context, the entire generative process fails (Barnett et al., 2024; Shao et al., 2023).

E. Advanced RAG Approaches

More advanced methods have been developed to address the shortcomings of Naive RAG, collectively named Advanced RAG. These systems focus on enhancing the retrieval process and refining the integration between retrieval and generation by employing pre-retrieval and post-retrieval strategies such as query rewriting, re-ranking, and context compression. Modular

RAG goes a step further by introducing iterative retrieval mechanisms that involve multiple cycles of retrieval and refining of information (Gao et al., 2024). This has contributed to a shift from traditional "Retrieve-Read" frameworks that used to leave the final LLM to do all the "heavy lifting" in a single hop (Ma et al., 2023). This evolution in RAG methods improves the precision of the retrieval process and enhances the overall quality of the generated content.

F. Multi-hop retrieval

A concept of Modular RAG is used in the SELF-RAG framework, which incorporates self-reflection mechanisms and on-demand retrieval, enabling it to evaluate the utility of retrieval results and decide when to retrieve additional information. A similar, but broader, concept is found in the ReAct framework, which stands for Reasoning and Acting, and allows the LLM model to perform actions based on insights from observations (Yao et al., 2023). These methods have been widely adopted by popular LLM frameworks such as Langchain and LlamaIndex in the form of agents with tools (Langchain, 2024d, 2024e).

G. Graph RAG Approaches

The improvements of Advanced and Modular RAG, in addition to multi-hop reasoning alone, are not sufficient to address all the limitations of current RAG systems. To further improve the effectiveness and reliability of RAG on large multi-document data, graph-based methods are actively being developed.

The KAPING paper by Baek et al. (2023) introduces a novel approach that leverages knowledge graphs for more effective retrieval. In this approach, similar triples are retrieved from the knowledge graph based on the entities present in the user query. These triples are then converted into textual representations, which play a crucial role in grounding the LLM in well-structured, fact-based information, addressing some of the challenges NaiveRAG systems face (Baek et al., 2023). Similarly, the G-Retriever framework exemplifies the process of triples retrieval by constructing a sub graph that pertains to the user query, taking neighbourhood information into account (He et al., 2024).

Wang et al (2023) carries the concept of Modular RAG into graph territory by introducing graph traversal module using prompting, that enables LLMs to reason over multiple documents in multi-hop question answering tasks (Y. Wang et al., 2023). However, this approach lacks granularity as it stores large semantic structures like pages and passages as individual nodes. Thus, a more narrow approach is to use LLM to construct the underlying knowledge graph, representing entities and concepts present in the text as nodes and existing relationships between them as edges (Meyer et al., 2024). Langchain framework has adopted many of these methods using tools such as LLMGraphTransformer, which performs automatic entity relationship extraction using LLMs with structured output (Langchain, 2024a).

H. LLM assisted Knowledge Graph creation

The creation of knowledge graphs and ontologies traditionally relied on manual curation by domain experts, making the process costly and time-consuming. However, with the advent of large language models (LLMs) capable of outputting structured formats and demonstrating strong generalization abilities, this task can now be performed with minimal human intervention.

The use of LLMs for knowledge graph creation has shown promising results, demonstrating the ability to extract meaningful information from unstructured data using models like GPT and REBEL (Meyer et al., 2024; Trajanoska et al., 2023). Yao et al. (2024) also demonstrates that LLMs can perform tasks such as triple classification and relation prediction quite accurately. Similarly, LLMs can also write structured queries in relevant Graph Query Language (Cypher for Neo4j) to retrieve sub-graphs pertaining to the context of the question (Shang & Huang, 2024). These developments suggest that LLMs can be utilised not only as generators but also as powerful tools for structuring and querying knowledge, expanding their utility beyond traditional text generation.

III. Methodology/Data Pipeline

Data handling in our hybrid RAG system can be broken down into two distinct stages: indexing and query processing.

A. Indexing pipeline

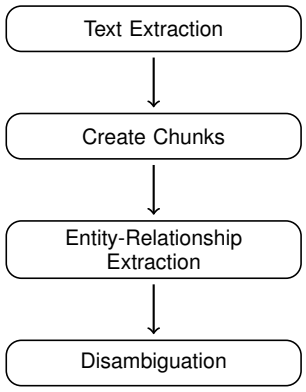

Fig. 1. Data Pipeline for Knowledge Graph Construction

The indexing process occurs when the data is first uploaded to the application and is performed only once per uploaded source. It begins with extracting raw text from PDF documents using appropriate libraries such as PyPDF and cleaning to remove formatting issues. The resulting structure, along with its metadata, is stored in the Neo4j graph database as a source node.

Once safely stored in a persistent state, chunking is performed, which splits the source text into manageable parts. It follows Langchain's implementation of the Advanced RAG, where chunks are stored and then subdivided into child chunks (**LangChainAdvancedRAG**). The system then embeds parent and child chunks, which get separate indexes. These chunks

are saved in Neo4j and are connected to the source node, which creates the following tree-like structure that ensures relational integrity:

```
(Document:Node)-[:HAS_PARENT]->
(Parent:Node)-[:HAS_CHILD]->(Child:Node)
```

Once the underlying structure for the document in the database has been constructed, the process of entity-relationship extraction is performed for each parent chunk using LLMGraphTransformer from Langchain (Langchain, 2024a). The general structure involves prompting OpenAI's GPT4 model, which has been configured with a detailed system prompt designed to extract notable entities present in the text and then create a list of triplets that form relationships between these entities. A validation process on the JSON output of the model is performed to make sure it is confined to the defined schema. Once the entity-relationship data has been extracted, it gets imported into the Neo4j database with every entity referenced to the appropriate chunk using [:PART_OF] relationship. Optional entity disambiguation procedure has been implemented using Tomaz Bratanic's work (Bratanic, 2024).

B. Query processing

1) Naive pipeline: This approach acts as a baseline for the study and employs a chunk-based retrieval method, using embeddings generated for context inside larger Parent nodes with 512 tokens.

2) Parent pipeline: The Parent RAG pipeline leverages both the Parent and Child nodes to perform similarity on embeddings from smaller Child nodes (100 token size) to then retrieve text stored in associated Parent nodes (Langchain, 2023).

3) Graph pipeline: Graph pipeline operates by first identifying and extracting key entities from the user's query using an LLM with the custom prompt. Once entities are extracted, the retriever queries the neo4j database using a full-text index to find matching entities inside the knowledge graph. Each identified entity node is then expanded to include its neighbouring entities that are within one degree of separation, capturing both outgoing and incoming relationships. The system then formats results into a textual representation of these directed triples to make it easily interpretable by the LLM.

4) Hybrid pipeline: At run time, when the user submits a query, two distinct retrieval methods are utilised (Figure 2). The Graph RAG pipeline performs entity searches inside the graph, providing structured information about the related concepts and other information. Simultaneously, the unstructured retriever, based on the Naive RAG, performs a similarity search on parent chunks. The system gathers metadata from its source node (such as document name) and its neighbouring entities for every retrieved parent chunk. These results are then augmented as context and passed to the LLM with the original user prompt.

5) Multi-hop approach: This paper goes further by using Langchain's agents to orchestrate ReAct logic to implement a multi-hop RAG approach (Figure 3). It wraps the Advanced Chain as a tool, making it available to the agent on demand (Langchain, 2024b). When a user first sends a query, the agent decides the sequence of actions to construct an answer. For example, the task involves writing an essay about Walt Disney. In that case, the agent will identify key details that need to be retrieved, such as his early life, notable achievements, influence on the entertainment industry, etc. It will then perform targeted action calls to the retriever tool to gather details on the aforementioned topics. After each call, the agent notes the resulting observation and refines its approach if needed. It notes down its chain of reasoning in a so-called "scratch pad", which helps guide its decision-making. Once the information is sufficient to cover all key details, the agent will synthesise

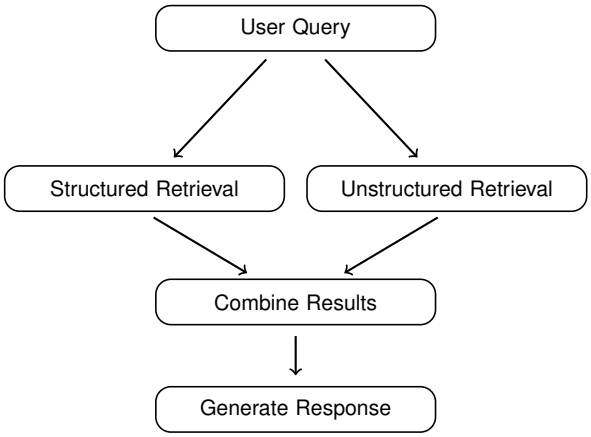

Fig. 2. Query Pipeline for Hybrid RAG System

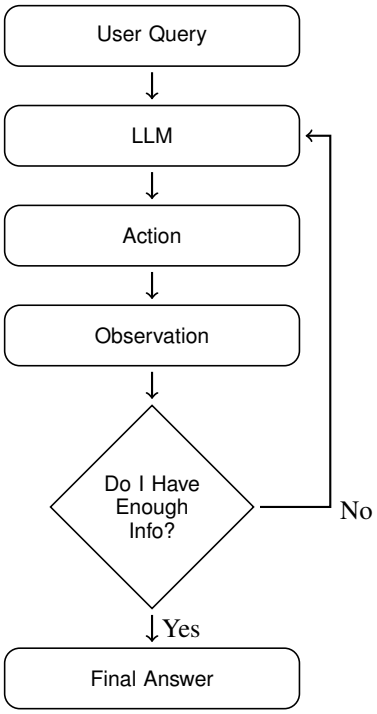

Fig. 3. Multi-hop Query Processing with Feedback Loop

the result from its ”scratch pad” to create a final answer.

IV. EXPERIMENTS

A. Datasets

Two synthetic datasets were created to evaluate the effectiveness of RAG pipelines in question-answering (QA) tasks. RAGAS evaluation framework was used to create a range of question-ground truth pairs with varied difficulty levels (Ragas, 2024b). Simple questions require only a single source of information to answer. Multi-context questions necessitate information retrieval from different sections or chunks to formulate an answer. Reasoning Questions require multi-hop strategies to derive the correct answer. The decision not to use well-established datasets such as Google's Natural Questions, TriviaQA, or WikiQA was primarily driven by resource constraints because these datasets contain thousands of documents and QA pairs, significantly elevating the project's cost.

1) Walt Disney Wikipedia: The first dataset is comprised of one PDF document with the content from Walt Disney's Wikipedia page and 20 contextually relevant QA pairs. This sets a good baseline to test the performance of indexing and query pipelines in generic QA tasks.

2) Scientific papers about RAG: Stepping up a notch, the RAG dataset contains knowledge from 10 academic papers about LLM and RAG research, which are cited in this study. Due to the larger size and complexity, 98 QA pairs were created to thoroughly test the system's performance in more specialised domains, offering a contrasting challenge to the more straightforward Walt Disney Wikipedia dataset.

B. Data indexing evaluation

The initial step in the experiment involved running datasets through the indexing pipeline and analysing the resulting graph structures. The Walt Disney dataset resulted in a graph containing 476 nodes and 660 relationships. It clearly identified the core entity of ”Walt Disney” and made coherent relationships, such as his affiliations to ”Disney Land” and his achievements in the industry.

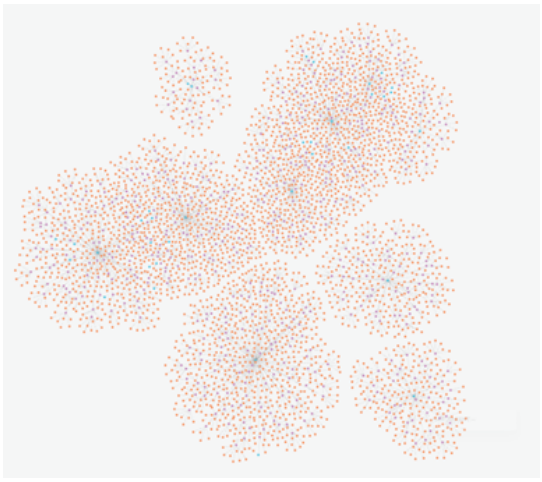

Fig. 4. Document–Parent–Child visualisation

Fig. 5. Parent–Entity–Entity visualisation

As expected, the RAG papers dataset yielded a much larger graph with 9202 nodes and 12264 relationships. Detailed visualisations were created using Neo4j Bloom to understand the resulting structure better. The Document – Parent-Child relationship diagram in Figure 4 highlights the arrangement of the content in a hierarchy, which can be used to traverse and extract information with varied levels of granularity. The Parent – Entity – Entity diagram in Figure 5 depicts the complex networks of interconnected entities between parent chunks. It was much more challenging to pinpoint and isolate a single node of interest on a graph. Therefore, a full-text query was performed to search for relevant concepts. For example, in the RAG dataset, querying graph for entity RAG returns the following triples:

```
(Rag)-[:ENHANCES]->(Llms)
(Rag)-[:COMPONENT]->(Retrieval)
(Rag)-[:COMPONENT]->(Generation)
(Rag)-[:VARIANT]->(Modular Rag)
(Rag)-[:APPLIED_TO]->(Open-Domain QA)
(Rag)-[:DEPLOYMENT_SOLUTION]->(Langchain)
(Rag)-[:DISCUSSION_TOPIC]->(Rag Challenges)
```

This highlights how a core concept of RAG was connected to entities originating from different sources. This observation is extremely valuable when retrieving neighbouring entities, as it can show insights that might not be immediately apparent when examining documents in isolation.

C. RAG pipeline evaluation metrics

Evaluating the QA performance of RAG pipelines on custom datasets presents unique challenges, partly due to the absence of golden answers and base performance to which to compare the pipelines. Thus, a head-to-head comparison was chosen to assess the models' relative performance. An analysis of both retrieved content and answers was carried out to get a complete picture. RAGAS evaluation framework offers a variety of metrics to assess the performance of pipelines (Ragas, 2024a):

- **Context Precision:** Calculates relevancy for each retrieved document and then takes the mean across all retrieved contexts.
- **Context Recall:** Calculates how many claims from the ground truth are present in the retrieved context.
- **Faithfulness:** Evaluates how many facts in the generated answer are faithful to the retrieved information, crucial for assessing the presence of hallucinations or inaccuracies in the final output.
- **Answer Relevance:** Measures how pertinent the generated answer is to the given question.
- **Answer Correctness:** Evaluates how accurately the generated answer reflects the ground truth.

It uses LLM as a Judge approach, which has shown to be effective at evaluating natural language generation tasks (J. Wang et al., 2023).

V. Conversational Platform Implementation

This section outlines the process of creating a conversational platform to demonstrate the capabilities and facilitate further development of graph-based RAG approaches described in the paper. A robust three-tier architecture consisting of a presentation tier that runs the user interface was chosen. The application tier performs all the RAG logic of the application, and lastly, a data tier oversees storing and retrieving data from knowledge graph (IBM, 2024).

A. Front end

The front end or presentation tier is built using Streamlit, a Python-based framework for creating interactive data-driven web applications (Streamlit, 2024). It was chosen for its simplicity and pre-made visual elements like chat and file uploader interfaces, as developing a custom JavaScript frontend with these features is beyond the project's scope. A multi-page layout of the app includes several key functionalities:

- Document upload using drag and drop method.
- Data Management for deleting individual documents stored in the knowledge graph.
- Multi Response Chat for simultaneous execution of multiple RAG chains
- Chat Agent for Multi-hop retrieval using ReAct pipeline.

During the design for the layouts of pages, special consideration was given to non-technical users by making the user interface (UI) as streamlined and straightforward as possible. The login screen features password visibility toggles and input validation to help prevent errors. It enables users to sign in to local or external/cloud neo4j instance to store their knowledge graphs. Robust error handling was implemented, providing simplified feedback to help users understand potential issues. An intuitive sidebar selection screen helps quickly navigate the app, and minimal graphical elements do not overload users with unnecessary information. The chat interface features a conversation history between the user and the AI, preserving responses in memory for the duration of the user's session.

B. Back end

The back end, or application tier, is constructed using FastAPI, a high-performance Python framework for building HTTP-based service APIs (Sebastián Ramírez, 2024). It was chosen because of support for asynchronous programming and native integration with Langserve, a module of Langchain designed to expose RAG chains as REST API endpoints (Langchain, 2024c). It provides multiple endpoints for document ingestion, data management, user query submission, and streaming of responses from RAG pipelines.

During development, various stages of data pipelines have been rewritten to support concurrency, freeing up the main event loop of the Fast API to allow the application to handle multiple user queries and document indexing. Slow tasks such as entity-relationship extraction have been parallelised using the ThreadPoolExecutor from Python's concurrent.futures module, which helped speed up the indexing of new documents dramatically. While Python's dynamically typed nature makes it very flexible in prototyping, malformed requests and other wrong-typed data can cause runtime errors that are hard to debug. The backend uses custom Pydantic models to mitigate these challenges and define a standardised schema for user credentials and API requests/responses. This ensures that data exchanged within the application is consistently validated and serialised to JSON format appropriate for HTTP transport (Pydantic, 2024).

C. Database

The application's data tier is implemented using Neo4j, an open-source graph database built in Java. It is one of the most popular production-ready graph databases, with many quality-of-life features. Neo4j uses Cypher query language, which shares some of its syntax with SQL but offers specific features to work with graph data, like pattern matching.

The deciding factor was its hybrid graph and vector storage capabilities and versatile Python API, which can support both Naive and graph-based RAG implementations without needing separate vector store (Neo4j, 2024). Neo4j offers extended procedures called Awesome Procedures On Cypher (APOC), which help perform complex queries and similarity/full-text searches, and its robust transaction management was crucial for implementing data operations like indexing multiple documents and running RAG chains in parallel.

D. Deployment

Docker was used to containerise the application components for deployment of the conversational platform, ensuring consistent environments across the development and testing stages. Docker Compose was employed to encapsulate the front end, back end, and Neo4j database. This setup simplifies service interactions through a shared network that is isolated from the host, increasing portability. With this setup, orchestration tools like Kubernetes can automatically scale and load balance the application, allowing for greater scalability. Lastly, a dockerised approach allows the app to be deployed on cloud platforms such as AWS and Azure, ensuring high availability.

E. External APIs

The system uses OpenAI API through the application to perform text embedding and natural language processing tasks using their GPT models. The decision to use OpenAI as an LLM provider and open source models boils down to their state-of-the-art (SOTA) performance. Several models, such as Llama 3 70b and Mixtral 8x7b, were considered. However, they incurred problems with structured output tasks and often produced malformed results, making them less reliable for the platform's requirements. That said, the application's design supports future transition to other models, supporting advancements in AI technology and reducing dependency on a single provider.

VI. RAG Evaluation Results

This section presents the evaluation results of RAG pipelines and conducts a critical analysis of the findings. Tables I and II reveal performance comparison between Naive, Neighbour, Graph, and Hybrid RAG approaches on our datasets with abbreviated metrics: Context Precision (C Prec), Context Recall (C Recall), Faithfulness (Faith), Answer Relevance (A Relev), and Answer Correctness (A Corr). In terms of content precision, Parent RAG scored the highest in both datasets (0.750 for Wikipedia and 1 for RAG papers), significantly outperforming Graph (0.012) and Hybrid RAG (0.054) on the Wikipedia dataset. However, these differences diminish in the RAG papers dataset. Hybrid and Naive RAG achieved the best performance in content recall with a tie on a smaller Wikipedia dataset (0.875). Still, Naive RAG took a slight lead on a more extensive knowledge base of RAG papers with 0.843 over Hybrid RAG with 0.844. In terms of faithfulness, Naive RAG performed best in both datasets (0.802 for Wikipedia and 0.939 for RAG papers), with Hybrid RAG coming close second with 0.799 and 0.865. Answer relevancy scores varied across the datasets, with Parent RAG performing best on Wikipedia (0.965) and Naive RAG on RAG papers (0.958). Hybrid RAG is slightly behind in second place for both (0.964 and 0.921). The answer correctness was the best for Parent RAG, with 0.593 on Wikipedia and 0.657 on RAG papers. Still, Hybrid and Parent RAG were also close to the mark.

TABLE I
Models Performance on Walt Disney Wikipedia

Model	C Prec	C Recall	Faith	A Relev	A Corr
Hybrid RAG	0.054	**0.875**	0.799	0.964	0.588
Graph RAG	0.012	0.135	0.567	0.382	0.364
Parent RAG	**0.750**	0.792	0.757	**0.965**	**0.593**
Naïve RAG	0.717	**0.875**	**0.802**	0.957	0.578

One notable observation across testing datasets is the underperformance of the Graph RAG approach. It struggles with context recall (0.135 for Wikipedia and 0.634 for RAG papers), indicating difficulty extracting enough relevant information. This and its low faithfulness suggest that the Graph RAG also suffered from hallucinations when inferring answers from limited relational information. Curiously, these deficiencies lessen in the larger RAG papers dataset.

TABLE II
Models Performance on RAG papers

Model	C Prec	C Recall	Faith	A Relev	A Corr
Hybrid RAG	0.916	0.843	0.865	0.921	0.633
Graph RAG	0.993	0.634	0.631	0.914	0.527
Parent RAG	**1.000**	0.668	0.859	0.864	**0.657**
Naïve RAG	0.988	**0.844**	**0.939**	**0.958**	0.649

Hybrid RAG exhibits similar behaviour on the Wikipedia dataset, with low context precision (0.054) but high recall (0.875). However, because it is also fed with structured information from affiliated chunks of text, it still generates relatively accurate answers. It is important to recognise that ineffective information extraction from graph structure is a two-way street and may be caused by inadequate retrieval and flaws in the construction of the knowledge graph. Additionally, when entity-relationship triples are passed as context, it impacts the metrics calculation, as the LLM judge deems them less relevant to the query than the full text.

Another observation is that Naive and Parent RAG exhibit inverse content precision and recall trends across both datasets. Parent RAG consistently scores higher in context precision, while Naive RAG excels in context recall. This suggests that Parent RAG is more effective at retrieving highly relevant documents, whereas Naive RAG captures a broader range of relevant content.

It is important to note that these results present preliminary comparisons of model performance; the resource constraints of this study made it impossible to perform a more thorough evaluation of larger QA datasets, which limits their generalisability and makes comparisons to approaches outside of this paper impossible. The absence of the golden standard evaluation methods for RAG further complicates forming definitive conclusions on performance. While RAGAS is a widely used evaluation framework, more research is needed into how its metrics translate to real-world performance. RAGAS failed to provide accurate metrics on several occasions despite the generated answer being technically correct. Thus, the LLM as a judge approach adds an element of unpredictability, with subtle differences in phrasing, context, or inherent biases within the model throwing off the results.

VII. Conclusion

This study demonstrates the potential of hybrid Retrieval-Augmented Generation (RAG) systems that combine knowledge graphs and vector search techniques to enhance the performance of LLM driven conversational agents. It offers a detailed overview of theory and implementation of both Naïve RAG as well as Graph RAG approaches, which culminated in the creation of Hybrid RAG pipeline, capable of harnessing the strengths of both vector implementations and relational knowledge graphs. This research extends its contributions by

developing a functional conversational platform called GraphBot, which leverages methods described in the paper. It serves as a practical demonstration of Hybrid RAG potential for real-world applications, and its modular architecture creates a robust test bed future research and development in the area. Its user friendly design also allows it to be tested by non-technical users such as domain experts in the field where it is applied.

VIII. Future Directions

While Hybrid RAG has shown comparable performance to Naive implementations, the results of the evaluation signal the need for further improvement. Future work could be focused on refining retrieval techniques, optimising knowledge graph creation, testing on well-established QA datasets while also considering cost and latency, exploring open source LLMs for indexing and generation and lastly, incorporating user testing and feedback to provide a more holistic evaluation of the RAG models' efficiency and practicality.

References

Baek, J., Aji, A. F., & Saffari, A. (2023). Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. https://arxiv.org/abs/2306.04136

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system.

Bratanic, T. (2024). Ms graphrag [Accessed: 2024-07-07].

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From local to global: A graph rag approach to query-focused summarization. https://arxiv.org/abs/2404.16130

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey.

He, X., Tian, Y., Sun, Y., Chawla, N. V., Laurent, T., LeCun, Y., Bresson, X., & Hooi, B. (2024). G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. https://arxiv.org/abs/2402.07630

IBM. (2024). What is three-tier architecture? [Accessed: 2024-07-12]. https://www.ibm.com/topics/three-tier-architecture

Langchain. (2023). Neo4j advanced rag [Accessed: 2024-07-06].

Langchain. (2024a). Constructing graphs with langchain [Accessed: 2024-07-06]. https://python.langchain.com/v0.1/docs/use_cases/graph/constructing/

Langchain. (2024b). Langchain agents documentation [Accessed: 2024-07-06].

Langchain. (2024c). Langserve documentation [Accessed: 2024-07-13]. https://python.langchain.com/v0.2/docs/langserve/

Langchain. (2024d). React agent types [Accessed: 2024-07-06]. https://python.langchain.com/v0.1/docs/modules/agents/agent_types/react/

Langchain. (2024e). React chat engine [Accessed: 2024-07-06]. https://docs.llamaindex.ai/en/stable/examples/chat_engine/chat_engine_react/

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the middle: How language models use long contexts. https://arxiv.org/abs/2307.03172

Ma, X., Gong, Y., He, P., Zhao, H., & Duan, N. (2023). Query rewriting for retrieval-augmented large language models. https://arxiv.org/abs/2305.14283

Meyer, L.-P., Stadler, C., Frey, J., Radtke, N., Junghanns, K., Meissner, R., Dziwis, G., Bulert, K., & Martin, M. (2024). Llm-assisted knowledge graph engineering: Experiments with chatgpt. In *First working conference on artificial intelligence development for a resilient and sustainable tomorrow* (pp. 103–115). Springer Fachmedien Wiesbaden. https://doi.org/10.1007/978-3-658-43705-3_8

Neo4j. (2024). Genai stack - neo4j labs [Accessed: 2024-08-14]. https://neo4j.com/labs/genai-ecosystem/genai-stack/

Pinecone. (2023). Vector similarity: An overview [Accessed: 2024-07-19]. https://www.pinecone.io/learn/vector-similarity/

Pydantic. (2024). Pydantic documentation [Accessed: 2024-07-14]. https://docs.pydantic.dev/latest/

Ragas. (2024a). *Metrics* [Accessed: 2024-07-14].

Ragas. (2024b). *Test set generation* [Accessed: 2024-07-14].

Sebastián Ramírez. (2024). Fastapi [Accessed: 2024-07-13]. https://fastapi.tiangolo.com/

Shang, W., & Huang, X. (2024). A survey of large language models on generative graph analytics: Query, learning, and applications. https://arxiv.org/abs/2404.14809

Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., & Chen, W. (2023). Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. https://arxiv.org/abs/2305.15294

Streamlit. (2024). Streamlit - an app framework for machine learning and data science [Accessed: 2024-07-12]. https://streamlit.io/

Trajanoska, M., Stojanov, R., & Trajanov, D. (2023). Enhancing knowledge graph construction using large language models. https://arxiv.org/abs/2305.04676

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. https://arxiv.org/abs/1706.03762

Wang, J., Liang, Y., Meng, F., Sun, Z., Shi, H., Li, Z., Xu, J., Qu, J., & Zhou, J. (2023). Is chatgpt a good nlg evaluator? a preliminary study. https://arxiv.org/abs/2303.04048

Wang, Y., Lipka, N., Rossi, R. A., Siu, A., Zhang, R., & Derr, T. (2023). Knowledge graph prompting for multi-document question answering. https://arxiv.org/abs/2308.11730

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). React: Synergizing reasoning and acting in language models. https://arxiv.org/abs/2210.03629